

Algoritmos e Estruturas de Dados II

Roteiro de Laboratório – Árvore Trie

Objetivo

Implementar Árvores Trie usando diferentes formas de gerenciamentos dos nós filhos: Hash, Lista Encadeada e Árvore Binária.

Pré-requisitos

- Compreensão dos conceitos de Árvore Trie, Tabela Hash, Lista Encadeada e Árvore Binária.

Atividades

Neste roteiro, o aluno deverá implementar árvores trie que armazenam cadeias de caracteres e aceitam prefixo. Como exemplo, armazenaremos nomes de países. Três diferentes técnicas de gerenciamento dos nós filhos utilizadas: tabela hash, lista encadeada e árvore binária.

Parte 1 – Árvore Trie com Tabela Hash

Nessa etapa, você irá implementar uma Árvore Trie em que os nós gerenciam seus filhos por meio de uma tabela hash. Basicamente, os ponteiros para os filhos de um nó são armazenados em uma tabela hash, cuja função hash é o código ASCII do caractere.

1. Criar a classe para a No

Criar a classe No para representar um nó da Árvore Trie. Os seguintes atributos devem estar presentes:

- a. char elemento: rótulo do nó;
- b. boolean folha: indicador de terminador de uma palavra;
- c. No[] prox: tabela hash que armazenará os ponteiros para os filhos do nó.

2. Criar construtor para a classe No

Criar o construtor da classe No, que recebe o rótulo do nó e configura o nó com o rótulo informado, terminador igual a *false* e sem filhos.

```
public No(char elemento);
```

3. Criar os métodos para classe No:

Criar os seguintes métodos na classe No:

- a. private int hash(char c): função hash para a tabela hash;
- b. public No getFilho(char c): retorna o ponteiro para o nó filho correspondente ao rótulo c;
- c. public void setFilho(No filho): configura o nó passado como parâmetro como um filho;
- d. public No[] getFilhos(): retorna um arranjo com todos os filhos do nó.

4. Criar a classe ArvoreTrie

Criar a classe `ArvoreTrie` para representar uma Árvore Trie. A classe possui como atributo um ponteiro para o nó raiz. O construtor da classe deve criar uma árvore vazia, isto é, deve criar o nó raiz com o rótulo de caractere vazio ('\\0').

```
public ArvoreTrie();
```

5. Criar inserção, pesquisa e caminhamento da Árvore Trie

Criar os métodos na classe `ArvoreTrie` para inserir, pesquisar e mostrar uma palavra na Árvore Trie. Os métodos são:

- a. `public void inserir(String s)`: insere a palavra `s` na árvore.
- b. `public boolean pesquisar(String s)`: pesquisa pela existência da palavra `s` na árvore. Retorna *true* caso a palavra esteja na árvore e *false* caso contrário.
- c. `public void mostrar()`: percorre a árvore e imprime todas as palavras existentes na árvore.

Como suporte aos métodos públicos acima, será necessário criar os métodos recursivos privados:

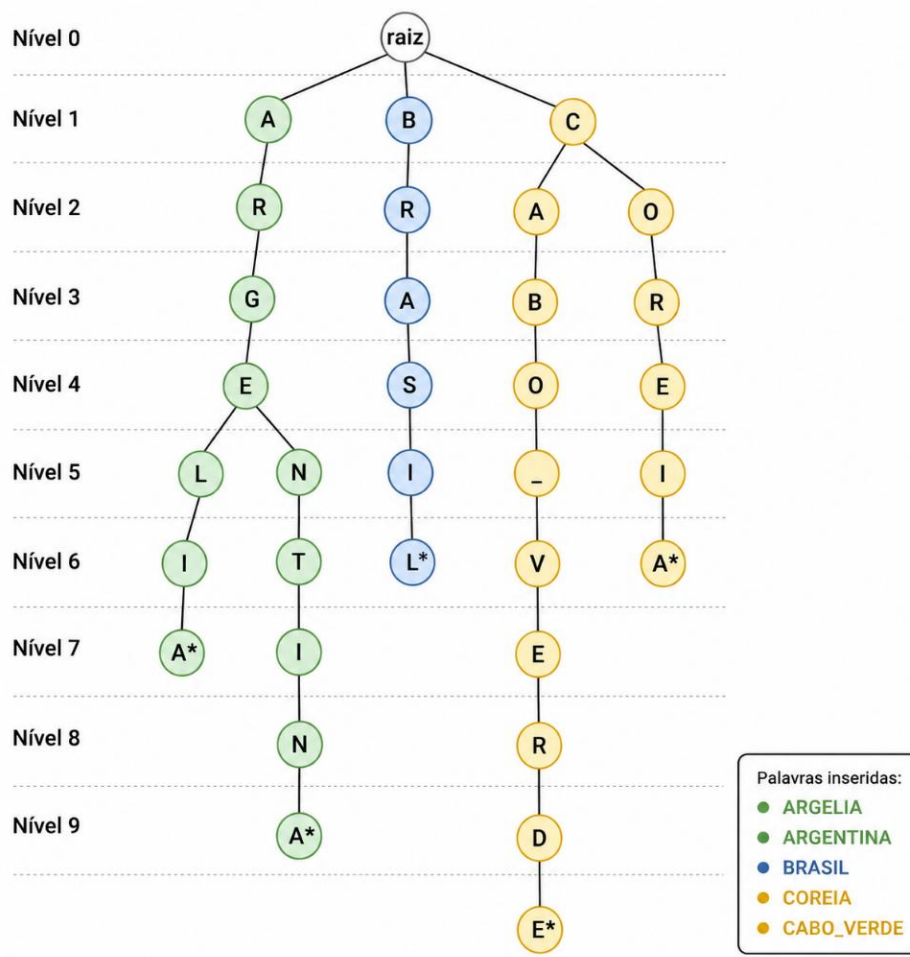
- d. `private void inserir(String s, No no, int i)`: onde `s` é a palavra a ser inserida, `no` é o nó sendo visitado e `i` o índice do caractere da palavra;
- e. `private boolean pesquisar(String s, No no, int i)`: onde `s` é a palavra a ser inserida, `no` é o nó sendo visitado e `i` o índice do caractere da palavra;
- f. `private void mostrar(String s, No no)`: onde `s` é a string sendo montada ao caminhar pela árvore e `no` o nó sendo visitado.

6. Testar a Árvore Trie

Criar um programa para validar a Árvore Trie. Seu programa deve realizar as operações e verificar se os resultados estão corretos. A sua estrutura de dados irá armazenar um conjunto de Strings representando países.

- a. Criar uma árvore vazia;
- b. Inserir um conjunto de países: BRASIL, ARGENTINA, ARGELIA, COREIA E CABO_VERDE;
- c. Mostrar todos as palavras da árvore;
- d. Pesquisar pelos países inseridos e certificar que a pesquisa retornou *true*;
- e. Pesquisar pelas palavras BRASILIA e MARROCOS e certificar que a pesquisa retornou *false*.

A figura a seguir exemplifica como ficaria a Árvore Trie com as palavras inseridas.



Parte 2 - Árvore Trie com Lista Encadeada

Repita o exercício da parte 1, implementando uma lista encadeada para gerenciar os filhos de um nó.

Parte 3 - Árvore Trie com Árvore Binária

Repita o exercício da parte 1, implementando uma árvore binária para gerenciar os filhos de um nó.